

# ProspectSoul — Technical-Team Developer Guide

**Version:** 0.1

**Status:** Developer Companion / Draft

**Applies to:** ProspectSoul v1

**Audience:** Product, technical team, developers, Claude, Claude Code, reviewers, and future maintainers

---

## 1. Purpose of This Guide

This document is the day-to-day technical guide for developing and maintaining ProspectSoul.

It does **not replace** the Product Requirements Document, Domain Model, Technical Design Specification, UI/UX Specification, or Implementation Roadmap.

The documentation hierarchy is:

1. **PRD v1.1** — what the product must do
2. **Domain Model v1.0** — business concepts and invariants
3. **Technical Design Spec v1.0** — technical implementation contract
4. **UI/UX Spec v1.0** — screens and interaction requirements
5. **Implementation Roadmap v1.0** — development sequence and acceptance criteria
6. **This Developer Guide** — practical engineering guidance and working conventions

When documents conflict, do not silently choose one. Stop and resolve the conflict.

---

## 2. Product Identity

ProspectSoul is Vyoog's internal **Lead Intelligence / Company Intelligence Repository**.

Its purpose is to transform raw prospect data into research-backed, ICP-qualified prospects before they enter the CRM.

The core workflow is:

```
graph TD;
  A[Import] --> B[Normalize];
  B --> C[Deduplicate];
  C --> D[ ];
```

```
graph TD; Research --> Review; Review --> ICP_Qualification[ICP Qualification]; ICP_Qualification --> Ready; Ready --> Export
```

Research  
↓  
Review  
↓  
ICP Qualification  
↓  
Ready  
↓  
Export

ProspectSoul ends where CRM/sales execution begins.

It is explicitly **not**:

- CRM
- Sales pipeline
- Opportunity management
- Calling system
- WhatsApp/email outreach system
- Marketing automation
- Web scraping engine in v1
- Customer-facing SaaS
- Multi-tenant product

These boundaries are part of the product requirements and must be treated as scope controls.

---

## 3. Architecture

### 3.1 Architectural Style

ProspectSoul is designed as a **modular monolith**.

There is:

- one Spring Boot backend
- one React frontend
- one PostgreSQL database
- one Keycloak integration
- one object-storage integration

The backend is divided into feature modules/packages rather than separate microservices.

The Technical Design specifies:

```
com.vyoog.prospectsoul/
```

```
config/  
common/  
company/  
contact/  
imports/  
triage/  
activity/  
evidence/  
research/  
icp/  
qualification/  
export/  
report/  
admin/  
ai/
```

Each feature follows the controller/service/repository/entity/DTO pattern where applicable.

## 3.2 Backend Principle

A business capability should normally live inside its own feature package.

For example:

```
company/  
    CompanyController  
    CompanyService  
    CompanyRepository  
    CompanyEntity  
    CompanyDto
```

Do not create a large generic `service/` package containing unrelated business logic.

Shared infrastructure belongs in:

```
common/  
config/
```

only when it is genuinely cross-cutting.

## 4. Frontend Structure

The documented frontend structure is:

```
frontend/  
└─ src/  
    ├── api/  
    ├── auth/  
    ├── components/  
    ├── pages/  
    └─ lib/
```

The specified technologies are:

- React 18
- Vite
- TypeScript
- React Router
- TanStack Query
- Ant Design, subject to substitution if Vyoog has an established house component library
- No Redux.

The UI is intentionally:

- desktop-first
- data-dense
- keyboard-friendly
- queue-oriented
- evidence-oriented

The Company Timeline is the heart of the application.

Do not turn the UI into generic CRUD screens.

The named screens in the UI specification are the intended screens.

---

## 5. Business Domain

### 5.1 Company Is the Anchor

The Domain Model defines:

Company  
├─ Contacts  
├─ Activities  
├─ Evidence  
├─ Qualifications  
├─ Exports  
└─ Aliases

Supporting concepts include:

Import Batch  
Import Rows  
Duplicate Candidates  
ICP Profiles  
ICP Versions  
Criteria  
Company Relationships (designed, not v1)

The one-sentence domain model is:

A Company accumulates Activities that produce Evidence, which supports a versioned Qualification, which gates an Export — and every record of all of it is permanent.

## 6. Pipeline State Model

Every company has exactly one pipeline state.

```
IMPORTED
  ↓
TRIAGE
  ↓
RESEARCH
  ↓
QUALIFICATION
  ↓
READY
  ↓
EXPORTED
```

Terminal states:

DISQUALIFIED  
ARCHIVED

They can be reached from the appropriate pipeline states and require a reason.

The state machine is a **server-side business rule**.

The UI must not be the only place enforcing it.

---

## 7. Core Domain Invariants

These are business rules, not UI suggestions.

### 7.1 One Company = One Real-World Entity

Duplicates must not silently coexist.

Merged identities retain aliases so old IDs can resolve to the surviving company.

### 7.2 AI Never Changes Business State

AI can suggest.

AI cannot:

- auto-merge
- auto-qualify
- auto-disqualify

Humans make those decisions.

### 7.3 No Evidence, No Claim

Every persisted AI claim must have:

- source
- URL or attachment
- verbatim excerpt
- capture date

An unsupported AI assertion is downgraded to `not_found`.

## 7.4 Qualifications Are Append-Only

A new qualification does not overwrite the old qualification.

Every qualification retains its ICP version.

## 7.5 Terminal Decisions Require Reasons

Disqualification, rejection and qualification overrides require reasons where specified.

## 7.6 Auditability

Every mutation must create an audit record.

The Technical Design specifies an explicit service-level:

```
AuditService.record(...)
```

rather than relying on magic listeners.

---

# 8. Import Architecture

v1 supports:

```
EXCEL_CSV  
INDIAMART  
TRADEINDIA  
LINKEDIN  
MANUAL_ENTRY
```

`API` exists as a reserved future source.

v1 is file/manual based.

The import contract is deliberately designed so future automated collection can become another source rather than a separate architecture.

## Import Rule

Every row must first be captured as an import row.

Nothing is silently discarded.

The system records:

- source
- batch
- raw row
- validation result
- eventual company/duplicate outcome

This provides lineage.

---

## 9. Normalization

Normalization is deterministic and unit-tested.

Documented rules include:

### Phone

Normalize:

```
+91 98430-12345
  ↓
9843012345
```

Invalid numbers are retained and flagged rather than silently discarded.

### Website

```
https://www.abc-pumps.com/products
  ↓
abc-pumps.com
```

### Company Name

Normalize:

- whitespace
- case
- punctuation
- configurable legal suffixes

Example:

```
ABC Pumps Pvt. Ltd.  
ABC PUMPS  
ABC Pumps Private Limited
```

should normalize to the same matching form.

## Location

Use the seeded reference list.

Aliases such as:

```
Madras → Chennai
```

are supported.

These rules are explicitly defined in the Technical Design Spec.

---

# 10. Deduplication

v1 uses deterministic matching.

Order:

1. normalized phone
2. website domain
3. normalized company name + city

A match creates a **duplicate candidate**.

It does not automatically merge.

Human decisions:

```
Merge  
Not a Duplicate  
Reject Row
```

Semantic/AI deduplication using pgvector is explicitly deferred until real data exists to tune it.

---

## 11. Database

### 11.1 Database

PostgreSQL is the v1 database.

The planning envelope is:

- 100k companies
- 300k contacts
- batches up to 10k rows

No premature scaling architecture is required.

### 11.2 Flyway

Flyway owns schema evolution.

Documented rule:

**Never edit an applied migration.**

Use:

```
V1__baseline.sql  
V2__seed.sql  
V3__...  
V4__...
```

Once a migration has been applied, create a new migration for the change.

Do not rewrite history.

### 11.3 Data History

Qualifications and merges are append-only.

The timeline is constructed from real tables.

There is deliberately **no second event store**.

This prevents the timeline from becoming a second source of truth.

---

## 12. API Conventions

The documented API base path is:

```
/api/v1
```

Conventions:

- UUID identifiers
- UTC ISO-8601 timestamps
- snake\_case JSON
- paginated list responses
- OpenAPI documentation through springdoc
- `/swagger-ui` enabled in non-production

Pagination:

```
?page=0&size=25&sort=created_at,desc
```

Response:

```
{
  "content": [],
  "page": 0,
  "size": 25,
  "total_elements": 0,
  "total_pages": 0
}
```

---

## 13. API Error Handling

Use:

```
application/problem+json
```

Documented error categories:

| HTTP | Meaning                 |
|------|-------------------------|
| 400  | Validation              |
| 401  | Unauthenticated         |
| 403  | Role denied             |
| 404  | Not found               |
| 409  | Conflict                |
| 422  | Business rule violation |

Example business-rule cases:

- invalid state transition
- duplicate resolution already completed
- override without reason
- disqualification without reason

---

## 14. API Idempotency

There is an important distinction here.

### Actually documented

The future programmatic source endpoint is designed around the CustSoul pattern:

```
POST /api/companies/from-source
```

with:

- service authentication
- source + reference idempotency

But this endpoint is **not part of v1 implementation**.

The PRD explicitly says it is deferred until a real programmatic source exists.

### Therefore

Do **not** invent an idempotency mechanism for v1 file imports unless the implementation specification is deliberately expanded.

When the first programmatic source is introduced, define and document the idempotency contract before implementing the endpoint.

---

## 15. Keycloak Authentication

The documented architecture uses Keycloak OIDC.

Realm:

```
vyoog
```

Clients:

```
prospectsoul-web  
prospectsoul-api
```

Roles:

```
PS_ANALYST  
PS_SALES_LEAD  
PS_ADMIN  
PS_VIEWER  
PS_COO
```

Backend:

- Spring Security Resource Server
- JWT validation
- roles mapped from `realm_access.roles`
- method-level authorization with `@PreAuthorize`

Frontend:

- `keycloak-js`
  - redirect login
  - silent refresh
  - token attached to API requests
-

## 16. Role Boundaries

| Capability          | Analyst | Sales Lead | Admin | Viewer | COO |
|---------------------|---------|------------|-------|--------|-----|
| Import              | ✓       | ✓          | ✓     | —      | —   |
| Triage              | ✓       | ✓          | ✓     | —      | —   |
| Research            | ✓       | ✓          | ✓     | —      | —   |
| Verify              | ✓       | ✓          | ✓     | —      | —   |
| Qualify             | ✓       | ✓          | ✓     | —      | —   |
| Tier override       | ✓       | ✓          | ✓     | —      | —   |
| Export              | —       | ✓          | ✓     | —      | —   |
| Configure           | —       | —          | ✓     | —      | —   |
| Read/Search/Reports | ✓       | ✓          | ✓     | ✓      | ✓   |

Authorization must be enforced server-side.

---

## 17. Background Processing

No message broker is required for v1.

The Technical Design uses PostgreSQL-backed queues and Spring scheduling.

### Import Processor

Triggered after batch upload.

Processes rows independently.

A bad row must not fail the entire batch.

### Research Worker

Scheduled approximately every 30 seconds.

Uses database locking:

```
FOR UPDATE SKIP LOCKED
```

to claim queued research work.

It respects:

- batch size
- daily AI cap
- per-company failure isolation

## Staleness Flagger

Runs nightly.

Recomputes stale status according to the configured threshold.

---

# 18. AI Architecture

AI is deliberately provider-independent.

The architecture defines:

```
public interface AiCapability<I, O> {  
    O execute(I input);  
    String capabilityName();  
}
```

v1 capabilities:

```
CompanyResearchCapability  
  
QualificationSuggestCapability
```

The provider registry maps:

```
capability  
  ↓  
provider  
  ↓  
model
```

↓  
prompt\_version

Configuration is DB-backed rather than hardcoded.

---

## 19. AI Provider

The v1 provider specified in the Technical Design is:

Anthropic Claude

called through REST.

However, the architecture should not make the business domain depend directly on Anthropic.

The domain talks to the capability interface.

The provider implementation sits behind it.

This permits later provider changes without rewriting business logic.

---

## 20. Website Research

The backend—not the AI provider—is responsible for fetching websites.

Specified component:

WebsiteFetchService

Technology:

Jsoup

Rules include:

- 15-second timeout
- maximum configured page set
- home

- about
- products
- contact
- certifications
- robots.txt respected

The fetched text and source URLs are passed to the AI.

This is important because the AI must cite real source URLs.

---

## 21. AI Output Handling

AI output is structured JSON.

It is parsed defensively.

A claim without:

source URL + excerpt

must not become an accepted evidence-backed claim.

It is downgraded to:

not\_found

This invariant is enforced by application code.

Do not rely on prompt instructions alone.

---

## 22. AI Logging

Every AI call is documented to record:

- capability
- provider
- model
- prompt version
- tokens
- duration

- success/failure

This creates traceability for AI behavior and cost.

---

## 23. Prompt Architecture

The documented architecture requires prompt identity to be configuration-driven.

A capability is associated with:

```
Capability
Provider
Model
Prompt Version
```

Example conceptually:

```
CompanyResearchCapability
    ↓
Anthropic
    ↓
configured model
    ↓
prompt_version = X
```

### Important Documentation Boundary

The supplied technical documents **do not contain the final literal production prompts**.

They define:

- capabilities
- inputs
- outputs
- provider abstraction
- routing
- prompt versioning
- evidence requirements

They do **not** define the complete production prompt text.

Therefore:

**Do not invent or silently create "canonical" production prompts based on this guide.**

Production prompts should become a separate controlled artifact when finalized.

---

## 24. AI Prompt Change Governance

This section is an engineering recommendation, not currently a fully specified project rule.

When production prompts are eventually created:

```
prompt/  
  company-research/  
    v1  
    v2  
  
  qualification-suggest/  
    v1  
    v2
```

A prompt change that can alter business output should:

1. receive a new prompt version
2. be tested against recorded fixtures
3. record the model/provider
4. preserve the old version for historical traceability
5. not silently alter historical AI results

This follows the documented requirement to persist `prompt_version`, but the exact repository convention is **not yet specified**.

---

## 25. ICP Architecture

ICP is configuration, not code.

An ICP contains:

- hard filters
- criteria
- weights
- thresholds
- tier boundaries

v1 has one configured ICP.

The data model supports multiple ICPs, but multiple active ICP UI is deferred.

Editing an ICP creates a new version.

Existing qualifications retain the old version.

---

## 26. Evidence

Evidence is a first-class domain concept.

An evidence record contains:

```
Claim
Source
URL or attachment
Excerpt
Capture date
```

Examples:

```
"Manufacturer"
  ↓
Products page
  ↓
verbatim excerpt
```

or:

```
"No ERP currently used"
  ↓
Research call
  ↓
call note
```

Evidence can support qualification criteria.

---

## 27. Activities

Activities currently include:

```
AI_RESEARCH  
MANUAL_NOTE  
CALL  
VISIT  
VERIFICATION
```

The scope guard is important:

**Research and verification only.**

Sales outreach such as:

- demos
- sales follow-ups
- negotiations

belongs in CRM.

---

## 28. Company Timeline

The Timeline is the default Company Detail view.

It is composed from existing tables.

It includes:

- imports
- merges
- activities
- verification
- qualifications
- state changes
- exports

There is no separate event store.

The purpose is to answer:

Have we ever looked at this company, and what happened?

quickly and without asking other people.

---

## 29. File Storage

Production:

S3 – Mumbai

Local:

MinIO

Same client abstraction.

Documented key patterns:

```
attachments/{company_id}/{attachment_id}/{filename}  
exports/{export_id}.csv
```

Allowed types:

```
pdf  
png  
jpg  
xlsx  
docx
```

Maximum:

20 MB

Downloads use short-lived pre-signed URLs.

---

## 30. Testing Expectations

Testing is not an afterthought.

## Unit Tests

Especially exhaustive for:

- normalization
- deduplication
- tier computation
- evidence enforcement
- state machine

These are explicitly identified as core product logic.

## Integration Tests

Use:

```
Testcontainers
```

with PostgreSQL.

The full pipeline should eventually be tested:

```
Import
→ Triage
→ Research
→ Qualification
→ Export
```

## AI Tests

Use:

```
MockAiProvider
```

in development and CI.

AI tests use recorded fixtures.

CI should not depend on live AI calls.

---

## 31. Definition of Done

The Implementation Roadmap defines the following for every sprint:

- all acceptance criteria pass
  - seed data demonstrates the behavior
  - unit tests cover business logic
  - integration test covers the sprint's main flow
  - audit rows are written for mutations
  - role restrictions are tested
  - OpenAPI matches implementation
  - README is updated
  - no sprint-scope TODOs remain
- 

## 32. Development Sequence

Do not develop features in arbitrary order.

### Sprint 1

Foundation:

- authentication
- Company
- Contact
- import
- search
- dashboard

### Sprint 2

Data integrity:

- normalization
- deterministic dedup
- triage
- activities
- timeline
- state machine

## Sprint 3

Intelligence:

- research queue
- AI abstraction
- AI research
- evidence
- attachments
- review

## Sprint 4

Decisions:

- ICP
- qualification
- reports

## Sprint 5

Handoff:

- Ready Pool
- export
- administration
- hardening
- full-pipeline test

The roadmap explicitly says:

Never start Sprint N+1 until Sprint N's acceptance criteria all pass.

---

## 33. Adding a New API

When adding an API:

1. Confirm it is required by the PRD/roadmap.
2. Check whether the endpoint already exists in the Technical Design.
3. Follow `/api/v1`.
4. Use UUIDs.
5. Use snake\_case JSON.
6. Use documented pagination conventions.
7. Use RFC-7807 errors.

8. Enforce authorization server-side.
9. Apply business invariants in the service/domain layer.
10. Record mutations in `audit_log`.
11. Add unit tests.
12. Add integration/API tests.
13. Update OpenAPI.
14. Update documentation.

If the required endpoint is missing:

**Propose it. Do not invent it silently.**

This is explicitly required by the implementation roadmap.

---

## 34. Adding a Database Change

1. Determine whether the change is actually required.
2. Create a new Flyway migration.
3. Never modify an applied migration.
4. Update the domain/entity model.
5. Update repositories/services as required.
6. Update API DTOs/contracts if exposed.
7. Add migration tests/integration tests.
8. Update seed data where necessary.
9. Verify existing migrations still execute from a clean database.
10. Verify existing tests still pass.

Do not add tables or fields merely because they might be useful later.

---

## 35. Adding a Feature

Before implementing a feature, answer:

What problem does it solve?  
Which pipeline stage does it belong to?  
Which domain entity owns it?  
Is it in v1 scope?  
What existing invariant does it affect?  
What API does it require?  
What database changes does it require?  
What UI screen owns it?  
What tests prove it works?

If those questions cannot be answered from the existing specifications, stop and clarify rather than improvising.

---

## 36. Adding AI Behavior

New AI behavior must follow this sequence:

```
Business requirement
  ↓
Capability definition
  ↓
Input contract
  ↓
Structured output contract
  ↓
Evidence requirements
  ↓
Provider routing
  ↓
Prompt version
  ↓
Recorded fixtures
  ↓
Mock provider tests
  ↓
Production provider
```

Do not put business rules solely inside prompts.

For example:

"AI must not qualify a company without evidence"

must also be enforced in application code.

---

## 37. Coding Rules

### Required

- Follow existing project conventions.
- Keep business logic inside the appropriate feature module.

- Keep shared utilities genuinely shared.
- Prefer configuration over hardcoding where the PRD specifies configuration.
- Validate on the backend.
- Keep API contracts explicit.
- Write tests for business rules.
- Keep migrations immutable.
- Keep auditability intact.

## Avoid

- speculative abstractions
- premature microservices
- generic CRUD replacing specified workflows
- silent API changes
- database schema shortcuts
- AI decisions without evidence
- bypassing state transitions
- UI-only authorization
- adding v2 features to v1

---

## 38. Claude Code Working Rules

Claude Code is the implementation agent.

The product/architecture specification remains authoritative.

Claude Code should:

1. Read the relevant specification before coding.
2. Work one sprint at a time.
3. Build vertical slices.
4. Never silently invent missing contracts.
5. Enforce business invariants in code.
6. Run tests after implementation.
7. Report deviations.
8. Stop when the sprint acceptance criteria are complete.

The roadmap explicitly defines these rules.

---

## 39. Claude Code — Implementation Prompt

Use this pattern when starting a sprint:

Read the ProspectSoul PRD, Domain Model, Technical Design Specification, UI/UX Specification, and Implementation Roadmap.

Implement Sprint N only.

Follow the documented API contracts and domain invariants exactly.

Do not begin Sprint N+1.

Before coding:

1. inspect the existing implementation,
2. identify the files that need to change,
3. identify any specification conflicts,
4. identify any missing contract that requires clarification.

Then implement the sprint as a vertical slice.

When complete:

- run all required tests,
- verify the sprint acceptance criteria,
- report every criterion as PASS/FAIL,
- report any deviation from the specifications,
- do not claim completion without evidence.

---

## 40. Claude Code — Review Prompt

After implementation:

Review the current ProspectSoul implementation against:

1. PRD v1.1
2. Domain Model v1.0
3. Technical Design Spec v1.0
4. UI/UX Spec v1.0
5. Implementation Roadmap v1.0

Do not redesign the product.

Check specifically:

- business invariants
- state transitions
- authorization

- API contract
- database migrations
- audit logging
- test coverage
- error handling
- UI workflow
- scope violations
- undocumented assumptions

For every finding provide:

Severity

Location

Requirement

Actual implementation

Gap

Recommended correction

Do not modify code during this review.

---

## 41. Git Governance

### Actually documented

The supplied implementation roadmap requires sprint completion and validation before moving forward.

### Not formally documented

The supplied ProspectSoul design documents do **not** define:

- branch naming
- commit naming
- semantic versioning
- release tagging
- pull-request rules
- merge strategy
- protected branches
- rollback/tag policy

Therefore these should not be presented as existing canonical ProspectSoul rules.

### Recommended working convention

Until formally documented:

```
main
  ↓
feature branch
  ↓
implementation
  ↓
tests
  ↓
review
  ↓
merge
```

Use milestone tags such as:

```
v0.0.0-bootstrap
v0.1.0-sprint1
v0.2.0-sprint2
v0.3.0-sprint3
v0.4.0-sprint4
v1.0.0
```

**This is a recommended governance convention, not currently a documented project requirement.**

---

## 42. PRD Generation Structure

The current PRD establishes a useful structure for future product requirements.

A new major product change should generally preserve:

1. Overview
  - ├ Problem
  - ├ What it does
  - ├ What it is NOT
  - ├ Success criteria
  - ├ Strategic direction
  - └ Key design decisions
2. Users and Roles
3. Core Concepts
4. System Architecture / Workflow

5. Data / Quality Rules
6. Operational Rules / Staleness / Controls
7. Search / User-facing behavior
8. Reports
9. Technical / AI architecture where required
10. Non-functional requirements
11. Out of scope
12. Open items

For ProspectSoul specifically, the PRD also records explicit deviations from brainstorming documents so the reasoning is not lost.

Future PRDs should distinguish:

- canonical requirement
- design decision
- deferred capability
- open question

Do not silently turn brainstorming material into requirements.

---

## 43. Scope Control — v1

The following are explicitly outside v1:

- automated scraping/collection
- programmatic source API
- semantic AI deduplication
- CRM API integration
- CRM conversion sync-back
- field-level verification
- numeric contact confidence
- multiple active ICPs
- ICP A/B testing
- third-party enrichment services
- GST/MCA lookups
- social-media enrichment

- automated re-research campaigns
- scheduled/emailed reports
- contact movement detection
- document intelligence
- natural-language search
- full contact lifecycle
- parent/subsidiary/company relationship implementation
- CRM functionality
- calling
- WhatsApp
- email outreach
- multi-tenancy
- customer-facing access
- lead marketplace/data resale

These are explicit scope controls, not suggestions.

---

## 44. Scope-Control Test

Before adding something, ask:

**Does this make the v1 workflow work better, or does it start building the future product?**

If it is future functionality, defer it unless the PRD is formally changed.

The architecture may support future capabilities.

That does not mean they belong in v1.

---

## 45. New Developer Onboarding

### Environment

- ☐ Clone ProspectSoul repository
- ☐ Install Java 21
- ☐ Install Node.js
- ☐ Install Docker
- ☐ No global Maven installation required; use `./mvnw`
- ☐ Configure local environment variables
- ☐ Start PostgreSQL/Keycloak/MinIO
- ☐ Verify backend starts
- ☐ Verify frontend starts

## Documentation

Read in this order:

- ☐ PRD v1.1
- ☐ Domain Model v1.0
- ☐ Technical Design Spec v1.0
- ☐ UI/UX Spec v1.0
- ☐ Implementation Roadmap v1.0
- ☐ This Developer Guide

## Architecture

Understand:

- ☐ Company is the domain anchor
- ☐ Pipeline states
- ☐ Import lineage
- ☐ deterministic deduplication
- ☐ human decision gates
- ☐ evidence model
- ☐ qualification versioning
- ☐ audit model
- ☐ Keycloak roles
- ☐ AI capability abstraction

## Before coding

- ☐ Identify the relevant sprint
- ☐ Read its acceptance criteria
- ☐ Identify relevant API contracts
- ☐ Identify relevant migrations
- ☐ Identify required tests
- ☐ Confirm the feature is within v1 scope

## Before declaring complete

- ☐ Unit tests pass
- ☐ Integration tests pass
- ☐ API behavior verified
- ☐ Authorization verified
- ☐ Audit records verified
- ☐ OpenAPI updated
- ☐ UI behavior verified
- ☐ Acceptance criteria demonstrated
- ☐ No scope creep introduced

---

## 46. Current Documentation Gaps

This section is deliberately explicit.

### A. Final Production AI Prompts

**Status: NOT DOCUMENTED**

The architecture defines:

- capabilities
- provider routing
- model
- prompt version
- structured outputs

It does **not** provide the final literal production prompt text.

Do not invent it.

---

### B. Exact Current Repository Implementation

**Status: PARTIALLY DOCUMENTED**

The Technical Design specifies the intended repository structure.

It does not document every implementation detail of the actual repository after bootstrap.

The current bootstrap was separately verified during development, but that verification is not part of the supplied design documents.

---

### C. Git Governance

**Status: NOT DOCUMENTED**

No canonical branch, commit, tag, release, or PR governance is defined in the supplied ProspectSoul documents.

Recommended conventions in this guide are therefore explicitly recommendations.

---

## D. Final CRM Export Format

**Status: OPEN**

The PRD explicitly identifies the target CRM and import format as an open item.

The default CSV export template is defined as a shipped default, but the actual CRM target is not finalized in the design documentation.

---

## E. Real Source File Formats

**Status: OPEN**

Real IndiaMART, TradeIndia, and existing Excel samples are still identified as inputs needed to build the real import mappings.

---

## F. Final ICP Sign-off

**Status: OPEN / CONFIGURATION**

A default ICP is defined for seed data, but the actual production Tier A/B/C criteria and thresholds require sign-off.

The architecture treats these as configuration, not code.

---

## G. Keycloak Production Configuration

**Status: OPEN**

The technical design defines the intended realm, clients, roles, and authentication mechanism.

The actual production realm/client/audience configuration still requires confirmation with the Keycloak operator.

---

## H. Attachment Storage Production Details

**Status: PARTIALLY DOCUMENTED / OPEN**

The architecture says S3 Mumbai in production and MinIO locally.

The PRD still identifies production bucket/configuration details as an open item.

---

## I. Research Queue Production Defaults

**Status: CONFIGURATION OPEN**

The architecture defines:

- batch size
- daily AI cap
- scheduled worker

but the actual production values require an operational decision.

---

## J. Formal Release Governance

**Status: NOT DOCUMENTED**

The roadmap defines sprint sequencing and Definition of Done.

It does not define a formal release-management policy.

---

# 47. What Developers Must Never Assume

Do not assume that because something is technically possible it is approved.

Examples:

```
"pgvector is already available"  
    ≠  
"Implement AI dedup now"
```

```
"CRM API would be better"  
    ≠  
"Build CRM integration"
```

"AI could extract brochures"  
≠  
"Implement document intelligence"

"We can support multiple ICPs"  
≠  
"Build multi-ICP UI"

Architecture extensibility is not feature authorization.

---

## 48. Final Engineering Principle

ProspectSoul should be developed as a controlled sequence:

```
Business Requirement
  ↓
Domain Model
  ↓
Technical Contract
  ↓
Implementation
  ↓
Automated Tests
  ↓
Acceptance Criteria
  ↓
Review
  ↓
Release
```

Never:

```
Idea
  ↓
Claude Code
  ↓
Code
  ↓
"We'll figure it out later"
```

The product is intentionally designed so that **AI accelerates engineering without becoming the source of architectural decisions.**

The PRD defines the product.

The Domain Model defines the business truth.

The Technical Design defines the implementation contract.

The UI/UX specification defines the user experience.

The Implementation Roadmap defines the sequence.

Claude Code implements those decisions and reports deviations; it does not silently redefine them.